

Хранение и версионирование данных

Серверы · RAID · YTsaurus · S3 · DVC

Что разберём

- Серверы, дисковые массивы и защита от отказов
- Сжатие данных без потерь
- YTsaurus: таблицы и распределённая обработка
- S3, DVC и Git LFS: хранение и версии

Серверы

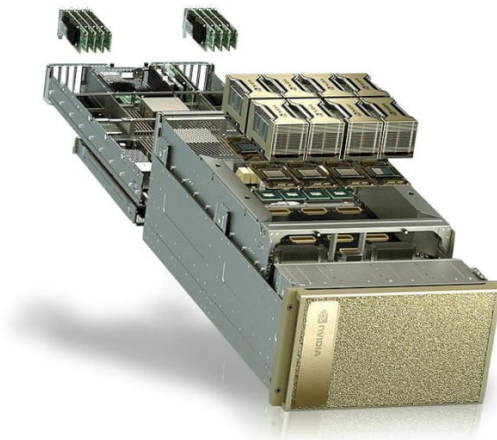
Роли серверов

Хранение



Диски и сеть

Вычисления



CPU, GPU, память

Сеть и кэш



Доставка данных

Что нужно вычислительному узлу

Ресурс

GPU

CPU: ядра, сокет

RAM: объём, каналы

PCIe

Каналы RAM $\neq$ линии PCIe

Задача

Обучение нейросети

Подготовка батчей

Данные процесса

Обмен с GPU и SSD

Что нужно узлу хранения

Ресурс

Задача

Диски

Ёмкость и скорость чтения/записи

Сетевые адаптеры

Передача нескольким клиентам

CPU

Контрольные суммы, сжатие

RAID: аппаратный контроллер или программная реализация

RAID

Дисковый массив RAID

RAID — Redundant Array of Independent Disks.

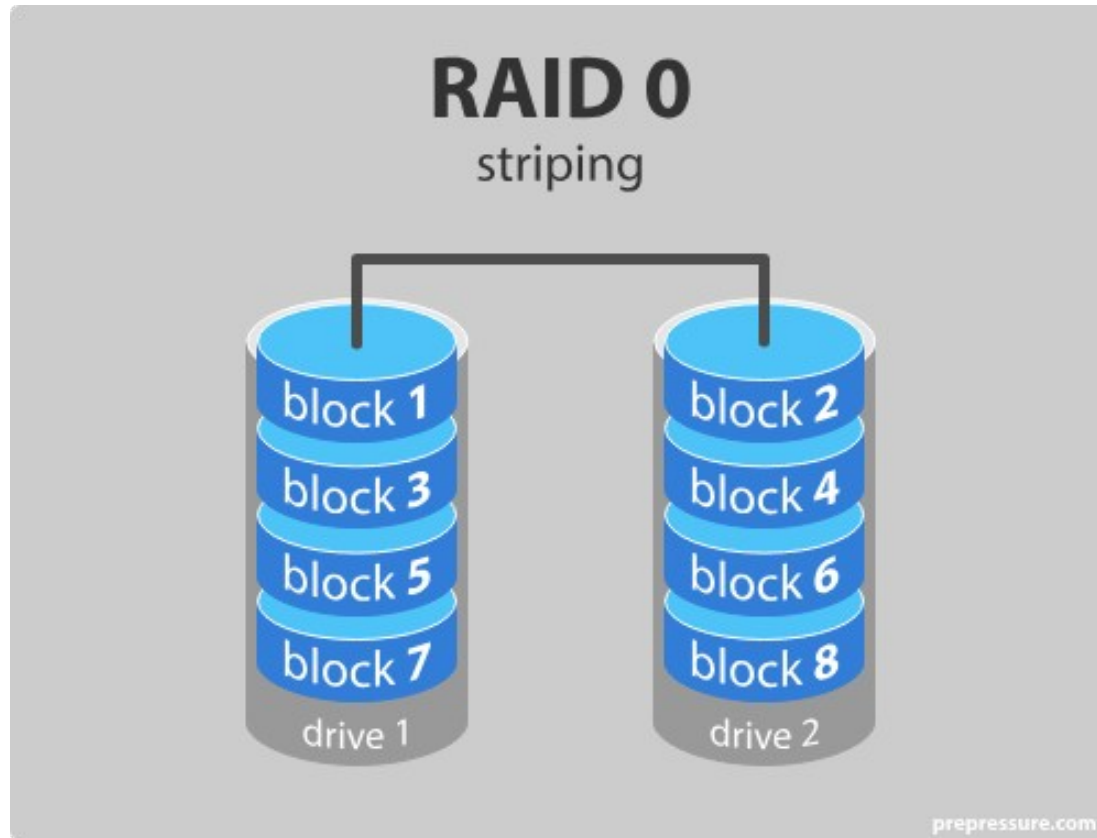
Компромисс: ёмкость, скорость, устойчивость к отказам.

N дисков по S : сравниваем полезную ёмкость.

Резервная копия — отдельно сохранённое состояние.

RAID

RAID 0: чередование блоков



Блоки распределены по дискам

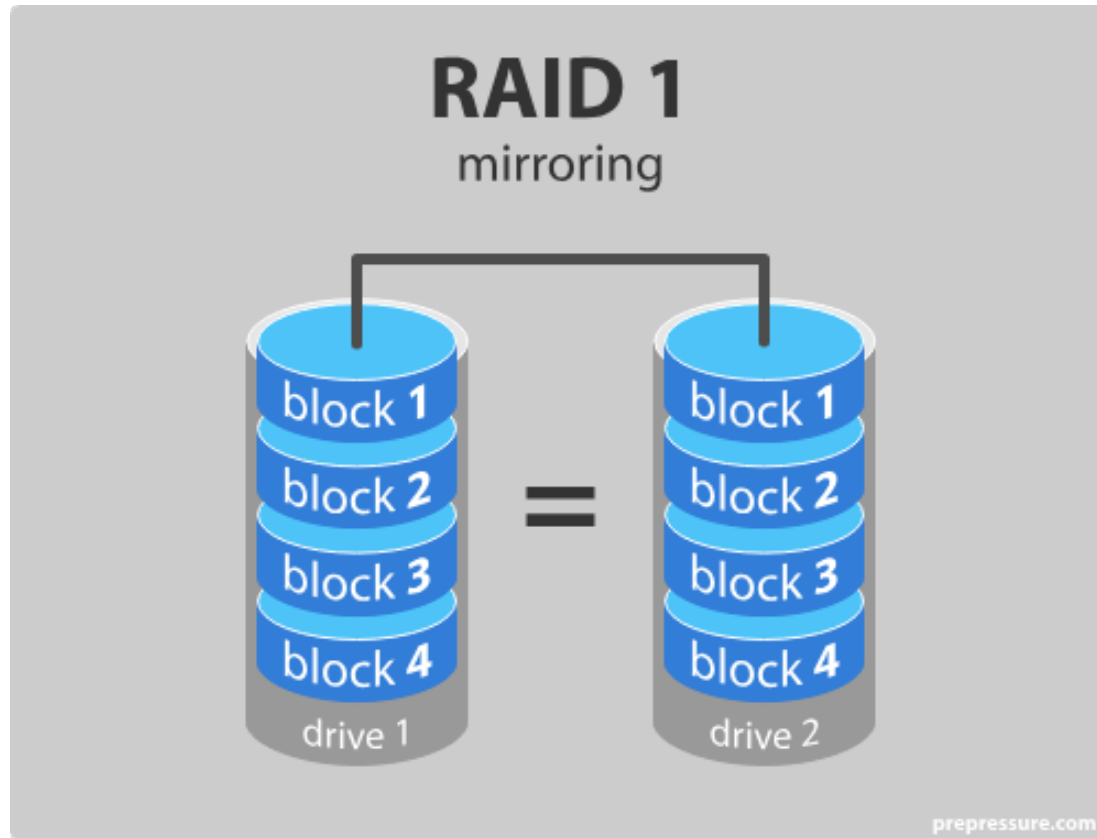
Ёмкость: $N \times S$

Допустимых отказов: 0

$2 \times 2 \text{ ТБ} \rightarrow 4 \text{ ТБ}$

RAID

RAID 1: зеркалирование



Два диска: одинаковые блоки

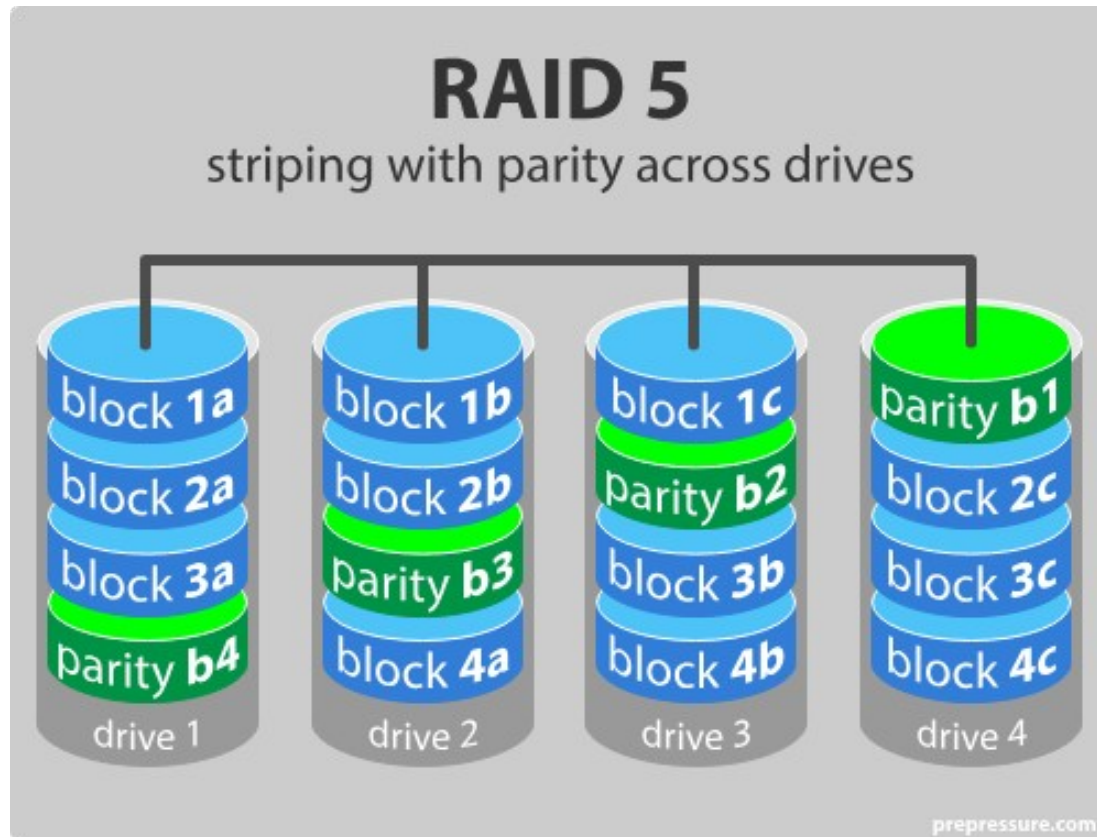
Ёмкость пары: S

Допустимых отказов: 1

$2 \times 2 \text{ ТБ} \rightarrow 2 \text{ ТБ}$

RAID

RAID 5: один блок чётности



Чётность восстанавливает блок

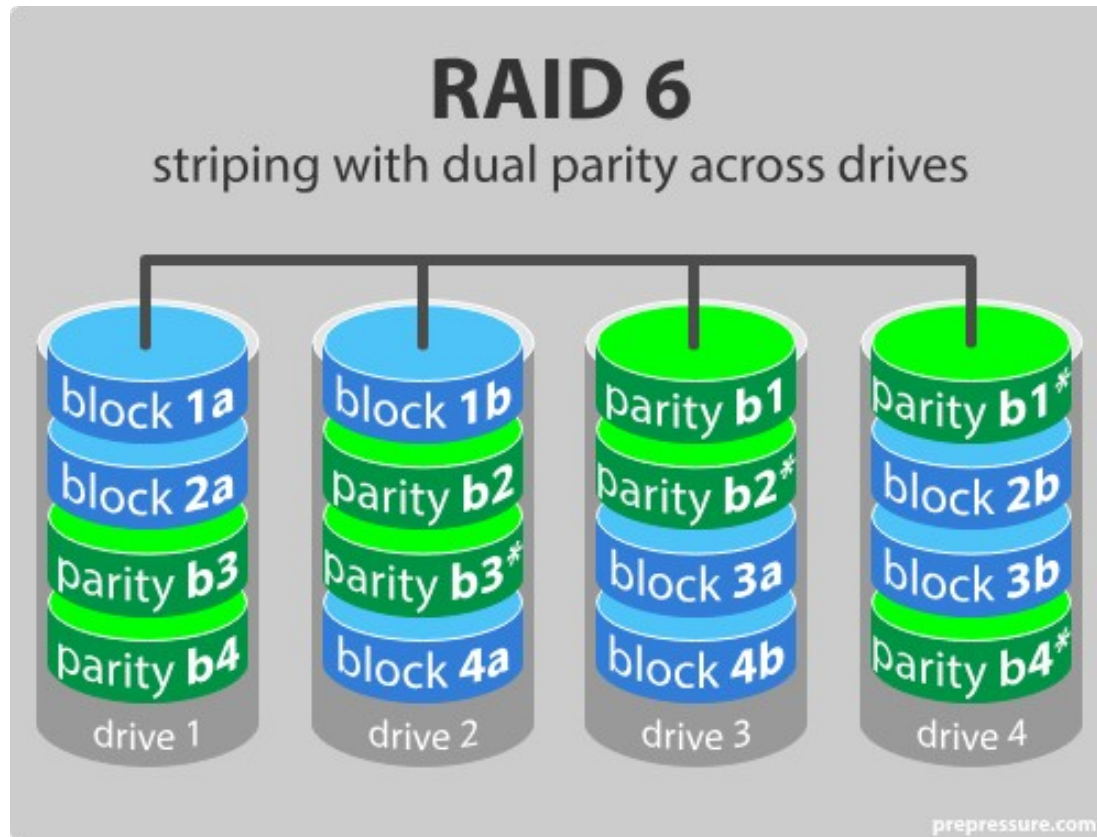
Ёмкость: $(N - 1) \times S$

Допустимых отказов: 1

$4 \times 2 \text{ ТБ} \rightarrow 6 \text{ ТБ}$

RAID

RAID 6: двойная избыточность



Два независимых проверочных блока

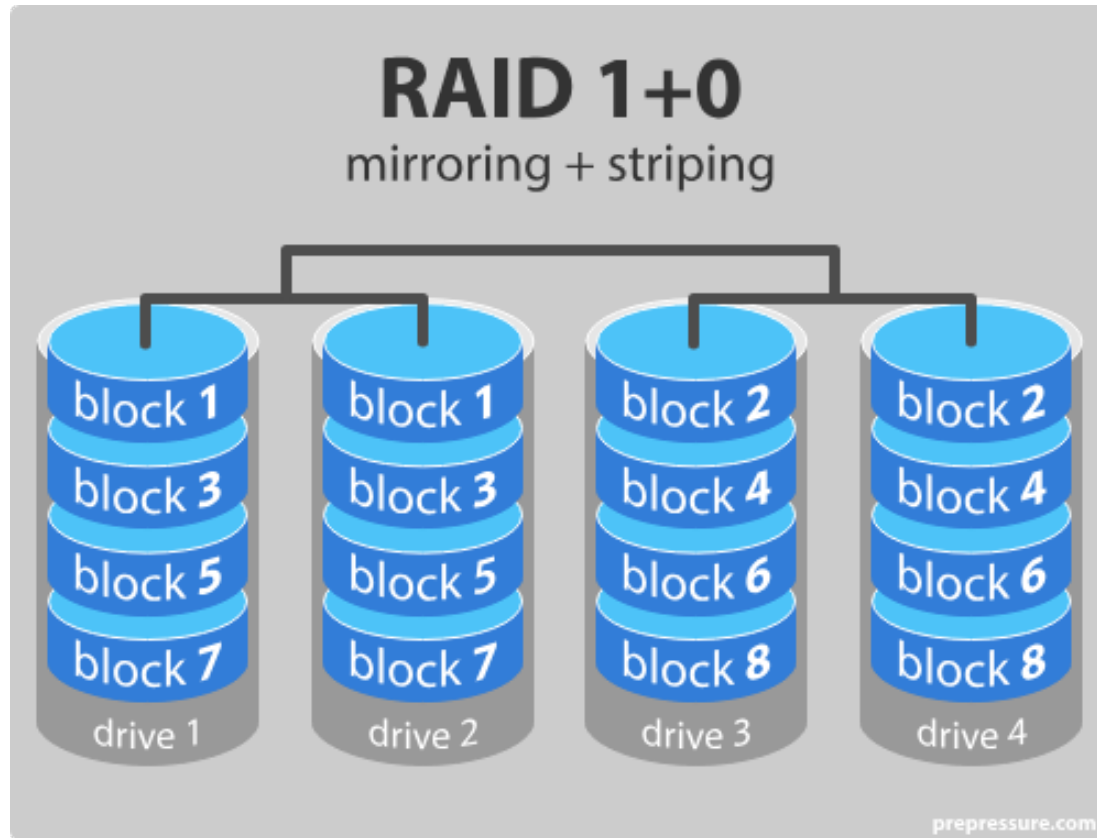
Ёмкость: $(N - 2) \times S$

Допустимых отказов: 2

$4 \times 2 \text{ ТБ} \rightarrow 4 \text{ ТБ}$

RAID

RAID 10: чередование зеркал



Блоки распределены по зеркалам

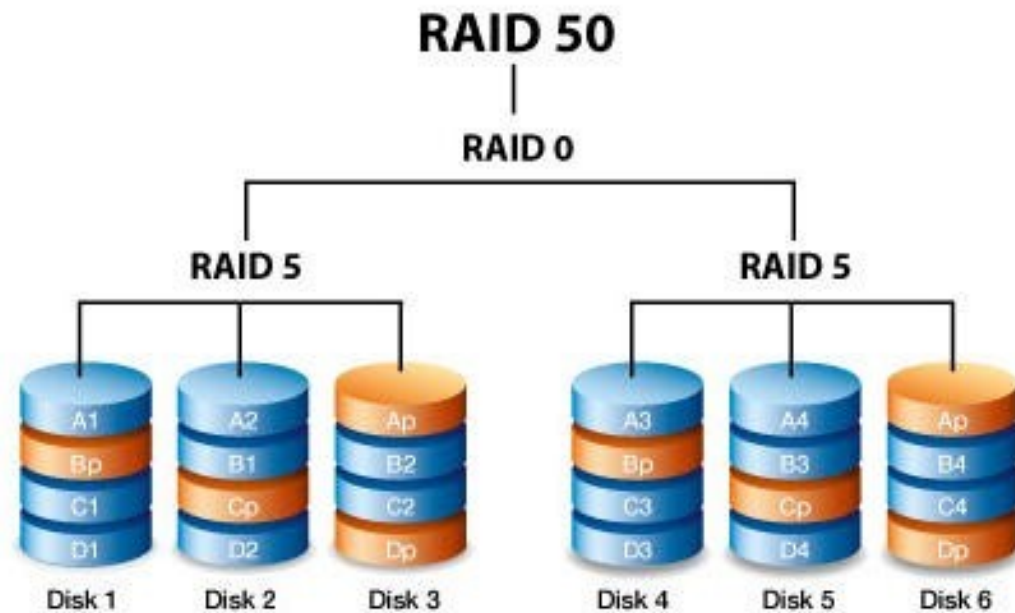
Ёмкость: $N \times S / 2$

Любой один отказ допустим

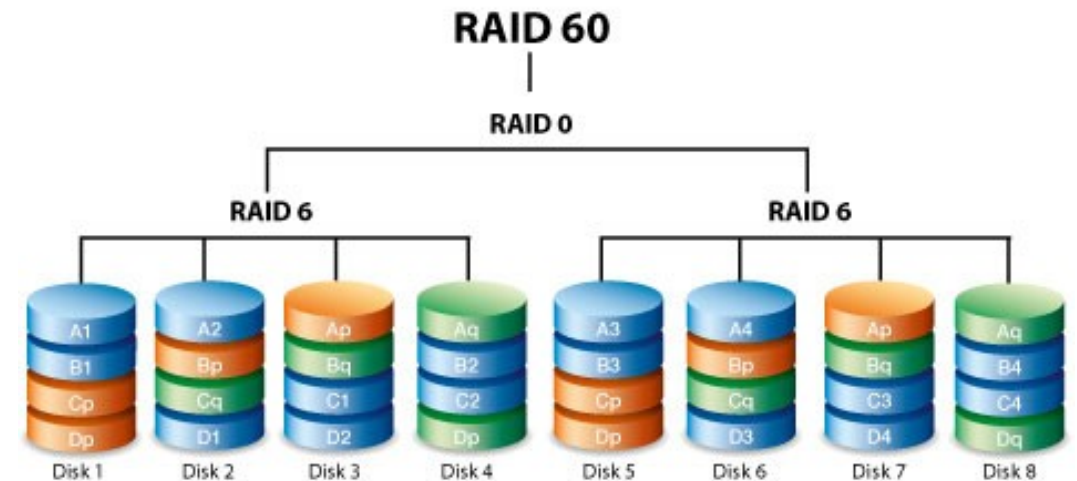
Два в одной паре — потеря

RAID

RAID 50 и RAID 60



До 1 отказа в каждой группе



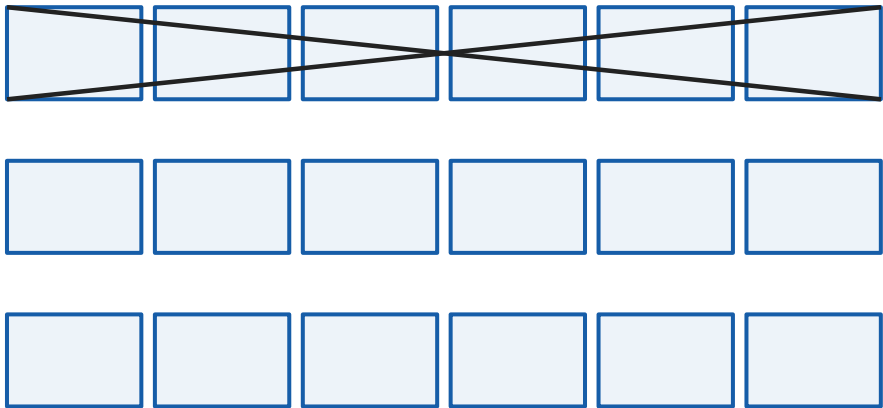
До 2 отказов в каждой группе

Репликация и избыточное кодирование

Исходные данные: 6 ГБ; клетка — 1 ГБ

× — недоступно

Репликация



3 копии → 18 ГБ

Erasure coding

6 частей данных



3 восстановительные



6 + 3 → 9 ГБ

Потерянный блок вычисляем из сохранившихся

Запись

A	B	P
1010	1100	0110

$$P = A \oplus B = 0110$$

Потеря A

A	B	P
????	1100	0110

$$A = P \oplus B = 1010$$

$\oplus$ — XOR: разные биты $\rightarrow 1$, одинаковые $\rightarrow 0$.

2+1: ≤ 1 потеря; Reed–Solomon 6+3: ≤ 3 .

[Ceph: erasure coding](#)

Как данные доходят от клиента до диска



Узел хранения: Сетевой адаптер → RAID → диски

При чтении: получить части → проверить → распаковать

Сжатие без потерь

Кодек — алгоритм кодирования и декодирования.

Байты → сжатие → хранение → распаковка → те же байты

```
import zlib
raw = b"id,value\n1,10\n" * 1000
packed = zlib.compress(raw)
assert zlib.decompress(packed) == raw
```

LZ4 · Snappy · Zstandard: размер ↔ скорость

[Zstandard](#)

Кодек выбирают по скорости и размеру

Кодеки

LZ4, Snappy

zlib, Zstandard

LZMA, bzip2

Brotli

Акцент при выборе

Скорость

Баланс скорости и размера

Степень сжатия

Текстовые данные

Сравниваем на своих данных; учитываем память.

YTsaurus: хранение и обработка



Распределённое хранение
и обработка данных

Клиент
Отправляет операцию



Кластер: группа серверов
Таблицы + вычисления

Результат сохраняется в кластере.

Возможности YTsaurus



Многопользовательская экосистема

- Набор связанных подсистем: MapReduce, движок SQL-запросов, планировщик, KV-хранилище данных для OLTP.
- Поддержка большого количества пользователей позволяет отказаться от лишних инсталляций и эффективно утилизировать железо.



Надёжность и стабильность

- Отсутствие единой точки отказа.
- Автоматическая репликация между серверами.
- Обновление кластера без потери прогресса вычислений.



Масштабируемость платформы

- До миллиона CPU и тысяч GPU.
- Эксабайты данных на разных носителях: HDD, SSD, NVME, RAM.
- 10 000+ узлов.
- Автоматический ввод и вывод серверов.



Обширная функциональность

- Расширенная модель MapReduce.
- Богатая транзакционная модель.
- Разнообразие SDK и API.
- Надёжная изоляция по вычислительным ресурсам и хранению.
- Удобный и красивый UI.



CHYT powered by ClickHouse®

- Привычный диалект SQL и знакомые функции.
- Быстрые аналитические запросы.
- Интеграция с популярными BI-решениями через JDBC и ODBC.



SPYT powered by Apache Spark

- Набор популярных инструментов для написания ETL-процессов.
- Запуск и поддержка нескольких кластеров SPYT.
- Простая миграция готовых решений.

Cypress: файлы, таблицы и ссылки

Cypress — дерево имён и метаданных.

Файл

Байты

Статическая таблица

Дописать или перезаписать

Динамическая (сортированная)

Обновить по ключу

Ссылка

Другой узел дерева

Чанк — неизменяемая часть хранения.

Обработка таблицы делится на задания

Тексты → токены

Scheduler

Планировщик

Выделяет

CPU и RAM →

Exec nodes

Вычислительные узлы

Controller agent

План операции

Назначает

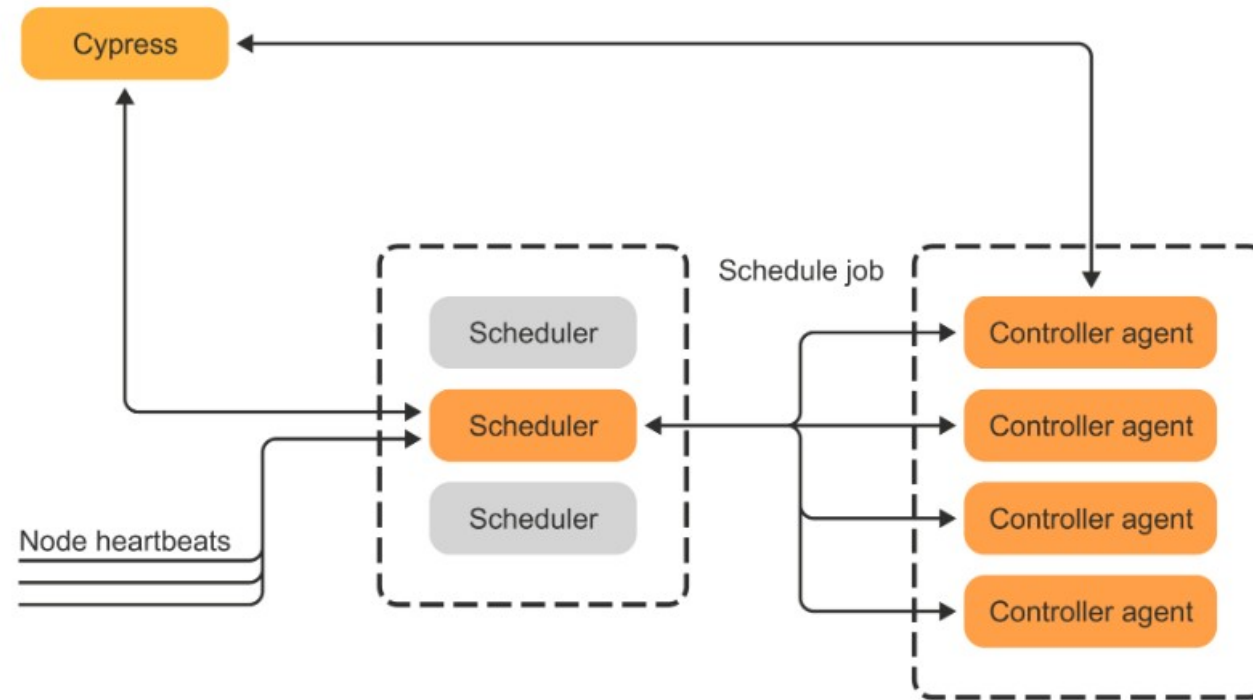
задания →

Часть А

Часть В

Job — отдельное задание операции.

Планировщик получает сведения о ресурсах



Heartbeat — сообщение узла о ресурсах и заданиях.

Scheduler: оранжевый — активный; серые — резервные.

[YTsaurus: планировщик](#)

Кластер распределяет ресурсы и переживает отказы

Требование

Отказоустойчивость

Масштабирование

Распределение ресурсов

Гетерогенность

Механизм

Повтор заданий

Новые узлы

Пулы, гарантии, лимиты

Разные CPU, RAM, GPU

Пул — группа вычислений с правилами распределения ресурсов.

Операции YTsaurus: от обработки строк до копирования

Map	Строки	Merge	Объединение
Reduce	Группы	Erase	Удаление диапазонов
MapReduce	Map + Reduce	Vanilla	Произвольная программа
Sort	Сортировка	RemoteCopy	Между кластерами

Интерфейсы: YQL · CHYT (ClickHouse) · SPYT (Spark)

GPU-задание должно переживать вытеснение



Вытеснение: планировщик останавливает задание

Новый запуск → загрузка чекпоинта → продолжение

Объектное хранилище S3

Amazon S3 — сервис объектного хранения.

`s3://course-data/data/docs.csv`

Bucket — контейнер
course-data

data/docs.csv

logs/day01.json
weights.npy



Key — имя объекта

data/docs.csv

Содержимое (фрагмент)

id,text

1,Установка Python

Метаданные

Content-Type: text/csv

Политика бакета определяет доступ к объектам

Кто	Действие	Объекты
Аналитик	Чтение	data/*
Обучение	Запись	checkpoints/*

ACL — список разрешений. В новых бакетах отключён.

Учебный пример правил; секреты не храним в Git.

S3

Журнал можно архивировать, затем удалить

Lifecycle: срок жизни (TTL) и смена класса хранения.

logs/day01.json



Версии сохраняют состояния, события запускают обработку

Версионирование

model.pkl

version A

version B

Выбираем состояние объекта

События

Загрузка объекта

↓

Уведомление

↓

Внешний обработчик

Уведомление может повториться.

Репликация копирует объекты в другой бакет

Режим

CRR: Cross-Region

SRR: Same-Region

Класс копии

Куда копируем

Другой регион

Тот же регион

Может отличаться

Копирование асинхронное: новая копия появляется не сразу.

S3

SeaweedFS читаем тем же S3-клиентом

SeaweedFS — S3-совместимое хранилище; boto3 — Python-клиент.

Amazon S3

```
import boto3  
s3 = boto3.client("s3")
```

Либо SeaweedFS: endpoint — адрес сервиса

```
s3 = boto3.client("s3", endpoint_url="http://localhost:8333")
```

```
s3.download_file("course-data", "logs/day01.json", "day01.json")
```

Ключи доступа задаются отдельно.

[SeaweedFS](#)

DVC

DVC: данные отдельно от Git

DVC (Data Version Control) — версионирование данных.

Git: data.dvc (фрагмент)

outs:

- path: data

- md5: cbc0b9be7f248d237b61fa2d586b2f08.dir

Remote

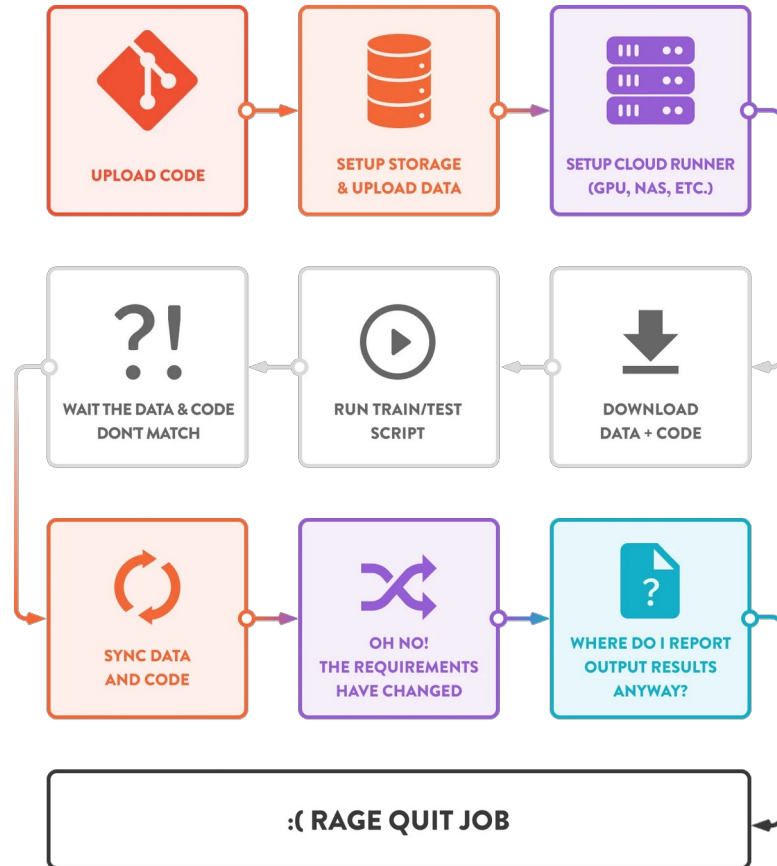
Общие данные

Указатель выбирает содержимое по контрольной сумме.

Кэш — локальное хранилище копий DVC.

[DVC](#)

Ручная передача не связывает код и данные

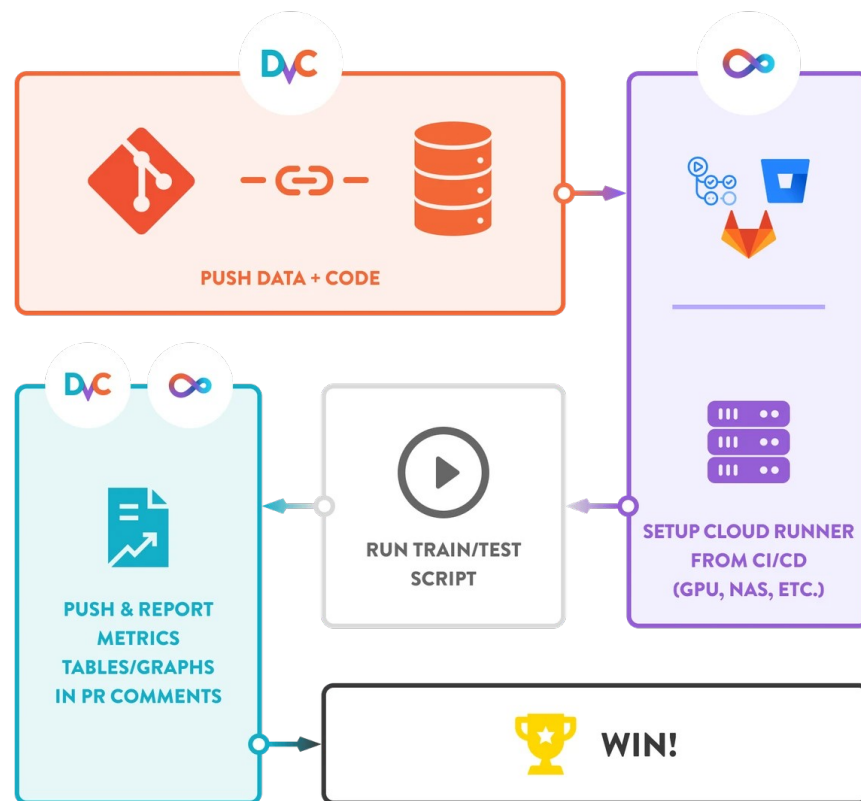


**Код и данные
передают отдельно**

При несовпадении версий
запуск приходится повторять

Runner — исполнитель скриптов

DVC фиксирует данные, CI запускает проверку

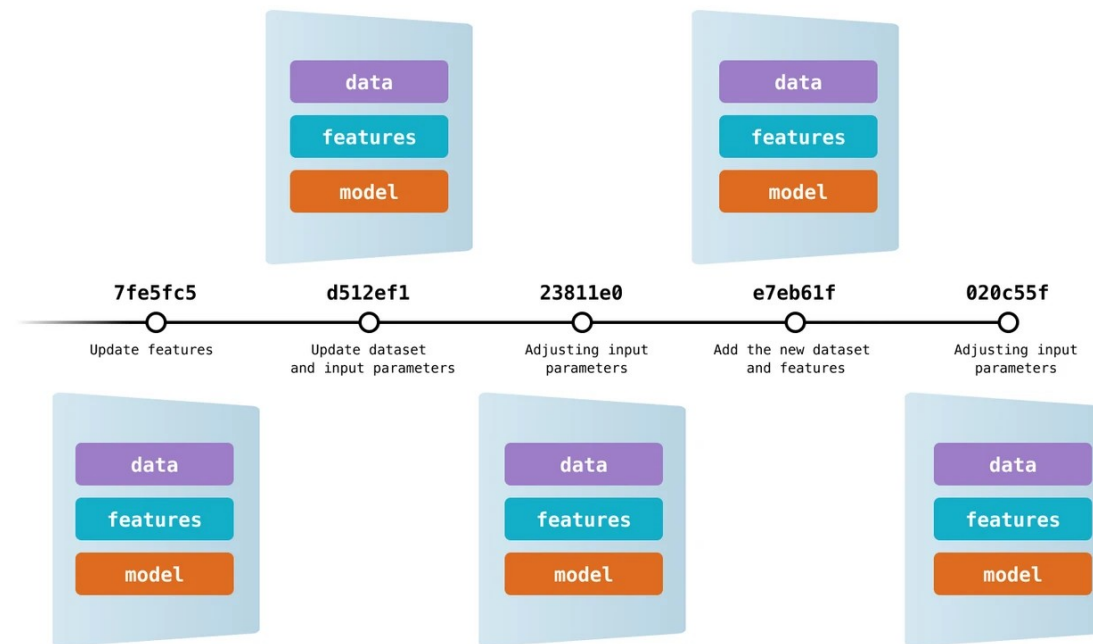


DVC — версии данных

CI — автоматические проверки

CML — ML-проверки и отчёты

Git-коммит связывает данные, признаки и модель



В Git — указатели; содержимое — в хранилище DVC.

[DVC files](#)

DVC хранит настройки, кэш и указатели отдельно

`dvc init`

`dvc remote add -d storage ../dvc-storage`

`git add .dvc .dvcignore`

`git commit -m "Configure DVC"`

`.dvc/config`

Настройки

`.dvc/cache/`

Содержимое файлов

Remote: локальная / сетевая папка,
SSH-сервер, S3 и другие хранилища.

`data.dvc`

Указатель после add

Отдельный сервер DVC не нужен.

[DVC: хранилища](#)

Передать данные коллеге

Автор

data.dvc



Git-репозиторий



data.dvc

data/



Хранилище DVC



data/

dvc add data

dvc push

```
git add data.dvc .gitignore
```

```
git commit -m "data v1"
```

```
git push origin main
```

[DVC](#)

Получатель

```
git pull
```

dvc pull

Вернуть предыдущую версию

data-v1 — 3 документа; data-v2 — 4 документа

```
git switch --detach data-v1
```

```
dvc pull
```

Шаг	Указатель	Документов
Исходно	v2	4
git switch	v1	4
dvc pull	v1	3

Git LFS: большие файлы в истории Git

Git Large File Storage — расширение Git.

Git: указатель. LFS-сервер: содержимое.

```
git lfs install --local
```

```
git lfs track "*.npy"
```

```
git add .gitattributes weights.npy
```

```
git commit -m "Add weights"
```

```
git push origin main
```

Отдельный пример: без DVC для этих файлов.

Итог

Какой механизм решает задачу

Задача

Пережить отказ

Сократить объём

Хранить и обрабатывать

Выбрать версию файлов

Ошибочное удаление: нужна резервная копия.

Механизм

RAID, репликация, кодирование

Сжатие

YTsauros; объекты — S3

DVC / Git LFS

Серверы, RAID, YTsaurus

[Red Hat RAID](#)

[Oracle: RAID](#)

[YTsaurus: обзор](#)

[YTsaurus](#)

[YTsaurus: обработка](#)

[Zstandard](#)

Материалы

S3, DVC, Git LFS

[AWS S3](#)

[S3 Replication](#)

[SeaweedFS](#)

[DVC](#)

[DVC pull](#)

[Git LFS](#)